

Bridging the Quantum Gap: A Robotic Modeling System for Visualizing Decoherence and Qubit Behaviors

Sruti Nallakukkala

Applied Physics Research Laboratory

Princeton International School of Mathematics and Science, Spring 2026

Abstract - Quantum computing is a rapidly growing field, applied in numerous fields because of its ability to solve significantly complex problems and calculate higher amounts of probabilities in shorter amounts of time, when compared to classical computers. However, while measuring the states of qubits to determine results, the measurement phase introduces quantum noise into the system, thus impacting the results from the qubit. Quantum noise compounds, and thus with each iteration, quantum noise causes the data to become increasingly inaccurate and unreliable for use in fields requiring high-precision results. In parallel with this issue, many primary and secondary students are not introduced to the field of quantum computing because of the advanced pre-requisites needed to be fulfilled and because of how education systems cannot change their curriculums as quickly. To support growing industries, however, the very fundamental component necessary is a population interested in pursuing the field and helping find new ways to fix existing problems or create new solutions and techniques. This project proposes the use of a robotic swarm to model qubit behaviors and interactions, with a model correcting the robots' "noise" after each calculation. In doing so, this project will create a cost-effective and engaging method of exploring a high-level, evolving field for a wider audience.

Index Terms - Quantum computing, quantum noise, STEM education, qubit modelling, robotic swarms

INTRODUCTION

Because of the rise in the applications of quantum computing, the field of quantum computing is growing exponentially. However, because of how quantum noise causes results measured and collected from each calculation to be inaccurate, the data received from quantum computers are often unreliable and unscalable.

Quantum error correction remains a significant problem because of how "[there] exists no principled means to reason about erroneous behaviors" causing "quantum programs almost certain[ly] to have errors" [3]. Because there are multiple materials and ways to create qubits, there are

equally many ways to correct quantum noise and decrease or even eliminate the impact the compounding nature of the error has on the overall results. Hence, researchers and developers created multiple methods, like applying additional magnetic fields, error bounds, or Quantum Error Correction (QEC) codes and AI models to find the error experienced by particular types of qubits [1; 3; 4; 5].

Furthermore, when comparing STEM education today to the developments occurring in STEM fields, many of the developments are not covered in the STEM curriculum of many schools. Because of which, many primary and secondary students are not given a solid introduction to developments in STEM, unless they decide to explore it themselves.

According to Bloom's Taxonomy, analyzing and evaluating are two of the highest forms of thinking and learning that allow students to understand what they learn. By analyzing and evaluating, students will be able to not only "[b]reak down information into parts" but they also will be able to "[j]ustify a decision or stand," allowing them to develop a deeper understanding for the knowledge they learn. Hence, because of the advantages of creating learning material in a way that harnesses the process of analyzing and evaluating, this project hypothesizes that by being able to create a tactile model and analogy to describe the behavior and interactions of qubits, students will be able to understand the concepts and workings of qubits better, even without pre-requisite knowledge [2].

The general purpose of this project is to create a tactile method to break down and explain fundamental concepts, such as quantum noise or quantum entanglement, involved in quantum computing. By using a robotic swarm, it is believed that because users will have a physical apparatus to interact with, they will be more engaged with the robots because of how they are able to directly interact with and influence the state and actions of the robots.

The overarching goals of this project are to translate qubit behavior and interactions into a physical model that users can interact with to see how their actions affect the state of the qubit exhibited by the robot; visually explain the impact quantum noise has on the results obtained from computations; and understand how tactile learning can help decrease barriers to learning about quantum computing.

PROJECT BRIEF

To make this project a success, the final product should be able to take the quantum states simulated through Qiskit (a package used to simulate quantum circuits) and be able to translate them into an action a robot does or multiple robots do.

Because there are multiple ways to introduce “noise” into the robotic swarm, this project involves the use of a model to correct the robots’ actions to be as close as the ideal action as possible. For example, if the ideal action was to rotate a robot’s head by 180 degrees but the simulation caused the robot to rotate its head by 98 degrees, the model would correct the rotation of the robot’s head by turning the robot’s head so that it rotates as close to 180 degrees as possible.

By including different mechanisms to introduce noise into the robot or into multiple robots, the model will be trained to be robust so that it can handle as many decoherence sources as possible.

As the robotic swarm will be used to represent multi-qubit behaviors and interactions, and each individual robot can either represent individual qubits or become a part of the swarm of robots representing multiple qubits in one system, when the swarm of robots are used to simulate multi-qubit behavior, the robots should also be able to act in a synchronized manner, to represent that they are “entangled.”

The robots should also be able to identify each other and understand what has been simulated in one robot and how they respond to the actions of that robot. Because of the property of entanglement, the actions of one robot will influence the actions of another robot. In this way, the robots will need to identify each other so that they act accordingly.

The methods used to create this project will be to begin understanding how to translate the Qiskit simulation results into a physical action the robots can do. As of now, because the current experimental set-up is an Arduino and a servo, the prototype will need to find a way to translate the results of measuring the state of the bit into an angle that allows the servo to rotate. Using the ideal values for the probability of attaining the state of $|1\rangle$, we can multiply this probability by the angle rotation range of the servo, thus getting the angle the servo needs to rotate. Because the code converts this number into an integer there are rounding errors, considered to be the “noise,” that cause the servo to turn an unideal amount. The gap between the ideal amount and the actual amount the servo rotates can now be considered to be the decoherence of the servo. Using a Python class, the Qiskit software, and an Arduino sketch, the servo is able to first simulate the result it needs to represent and use the result to find the angle it needs to rotate.

In this project, because it requires the use of discrete components, components that do not have proper quantum properties, the project’s prototypes and final products will never truly be able to simulate qubits. Furthermore, because the components used in this project are mainly electrical

components, the components will tend to build up internal resistances and dissipate energy in some form, causing unaccounted noise because the components are assumed to be ideal for the purposes of the project. Because the simulation itself, also considered to be ideal, also has noise involved in it while measuring the state of the qubit in the quantum circuit being examined, there is no true ideal that can be used to compare the prototype’s accuracy.

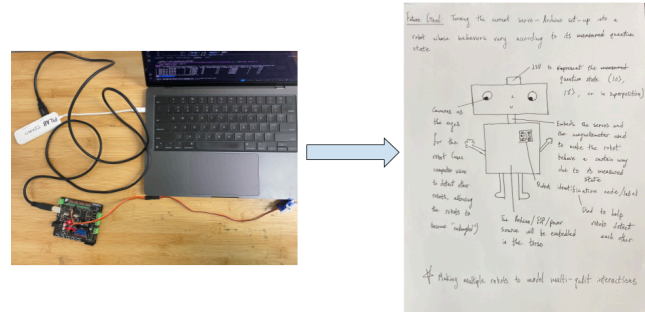


Figure 1: The current set-up versus the future goal for how the project should look like

As of present, the prototype involves the use of an Arduino and a servo. Because the Arduino and servo are the physical analogy for how a qubit might show its result as it is being measured, the project required the use of ideal angles for the servo to rotate. For each type of gate, there is an ideal angle the servo needs to rotate and the ideal probability that the simulation yields a result of $|1\rangle$ when the qubit it simulates is being measured.

```
self.IDEAL_VALUES = {
    'h': {'prob': 0.5, 'angle': 90},
    'x': {'prob': 1.0, 'angle': 180},
    'id': {'prob': 0.0, 'angle': 0},
    'sx': {'prob': 0.5, 'angle': 90},
    'y': {'prob': 1.0, 'angle': 180},
    'z': {'prob': 0.0, 'angle': 0}
}
```

Figure 2: A list of the ideal probabilities of the state of $|1\rangle$ and the ideal angles the servo needs to rotate

Using the list of ideal probabilities and angles, the code works to compare between the ideal and experimental results to calculate errors. Furthermore, because we can see the experimental result the code calculates, we can identify the error between experimental and ideal results to continue increasing the accuracy of the prototype in order to be able to more efficiently, accurately, and precisely model a qubit using discrete components. These comparisons allow us to

check the accuracy of the current model.

After using these comparisons and understanding the error found in my prototype, we can then work on bridging the gap between the experimental and ideal results. Since this is only the first stage of prototyping an experimental set-up, there are not as many factors needed to be changed in order to influence the results. As the number of components increase though, because the number of factors will increase, there will also be a significant increase in the number of factors needed to be changed in order to bridge the gap between the ideal and experimental set-up.

Because the current prototype used was intended to test the extent to which a servo can model qubit behavior when a qubit's state is being measured, the current prototype involves mainly the Arduino-servo hardware, the Python file allowing for the simulation of quantum circuits, and the Arduino file to allow for the rotation of the servo.

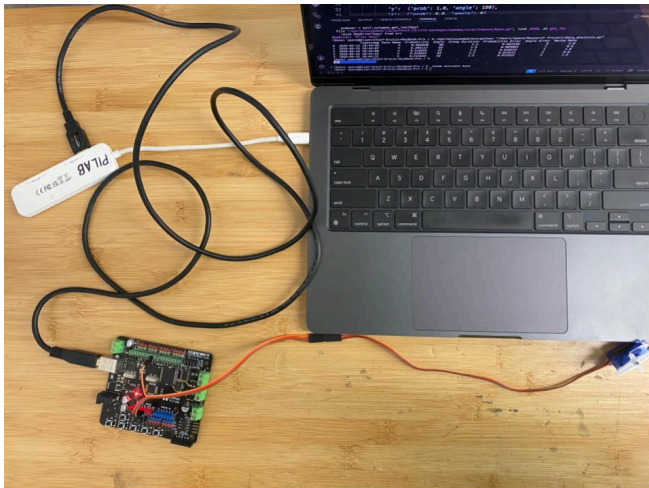


Figure 3: An image of the current experimental set-up for this research project

From Figure 3, seen above, the main part of this prototype relies on the software as the Arduino-servo set-up cannot model qubit measurement behavior on their own. Created by IBM, Qiskit, a library used to simulate and run quantum circuits, helps with running the simulation software for this prototype. Because Qiskit is Python-based, we can use Python code, along with Qiskit syntax, to program the actions the servo does. However, since the Arduino-servo set-up will not be able to do any actions without the C++-based code used for Arduinos, we can use libraries in Python in order to create a bridge between a Python program and an Arduino IDE program to let the servo rotate the experimental angle calculated by the Python program. In doing so, the Arduino IDE program is used to tell the servo to rotate the number of degrees sourced from the Python program. Below is a screenshot of the code used to allow the current prototype to successfully model qubit measurement

behavior.

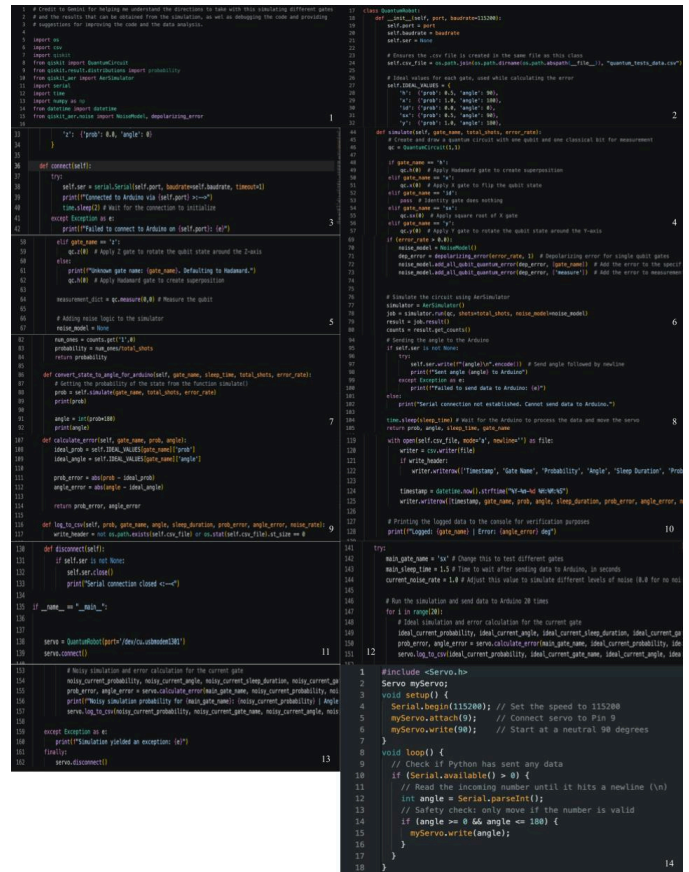


Figure 3: A collection of all the Python and Arduino IDE code used to help the Arduino-servo set-up successfully model the behavior of qubits as their states are being measured

REFERENCES

- [1] Abbasatarov, B., et al. (2025). Demonstrating magnetic field robustness and reducing temporal T1 noise in transmon qubits through magnetic field engineering. *arXiv*.
<https://doi.org/10.48550/arXiv.2506.02187>
- [2] “Bloom’s Classification of Cognitive Skills,” www.k-state.edu.
<https://www.k-state.edu/assessment/toolkit/outcomes/blooms.html>
- [3] Hung, S.-H., et al. (2019). Quantitative robustness analysis of quantum programs. *Proceedings of the ACM on Programming Languages*, 3(POPL), Article 31. <https://doi.org/10.1145/3290344>
- [4] Nachman, B., et al. (2020). Unfolding quantum computer readout noise. *npj Quantum Information*, 6, Article 84.
<https://doi.org/10.1038/s41534-020-00309-7>
- [5] Steane, A. M. (1996). Multiple-particle interference and quantum error correction. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*,

452(1954), 2551–2577. <https://doi.org/10.1098/rspa.1996.0136>

[6] Qiskit contributors, "Qiskit: An Open-source Framework for Quantum Computing," 2023. [Online]. Available: <https://qiskit.org>

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

APPENDIX A: ORIGINAL RESEARCH PROPOSAL

The original project proposal, detailing the initial plan for the project, as well as details on the fundamental quantum principles and how I must plan to model different types of qubits, as well as initial conceptual drawings are included here:

 Junior Research Proposal - Sruti N, Applied Physics ...

APPENDIX B: ARTICLE REVIEWS

The article reviews done to find more details about current works in Quantum Error Correction (QEC) are included here:  Article Review Annotations

APPENDIX C: SOURCE CODE

The code used for this project, including the simulations are included here:  Source Code